

Righteous IT *Join the crusade!*

Linux Investigation (Part 1)

Posted on May 4, 2026 May 4, 2026 by Hal Pomeranz

I recently posted a Linux scenario that I had mocked up and asked for people to submit write-ups for judging. The winners have now been announced

(<https://www.linkedin.com/feed/update/urn:li:activity:7457038612812197890/>). Congratulations to all who participated!

I also wanted to provide an analysis of my own. Obviously, I know exactly what happened because I did everything. But I'm going to approach this investigation as if I were coming in cold, without any prior knowledge.

The scenario (<https://righteousit.com/2026/03/27/linux-forensic-scenario/>) starts with an alert from the SOC containing two important pieces of information:

- Unencrypted traffic on port 22/tcp, specifically the string `"python3 -c 'import pty; pty.spawn("/bin/bash")'"`
- Heavy CPU usage but no process can be seen consuming the CPU

We have a UAC collection from the machine (https://deerrunassoc-my.sharepoint.com/:u:/g/personal/hal_deer-run_com/IQAGrMUqVLwEQ4Ceus_70Pn0AVxMaWSs7POONKw6ss103Bc?e=4N5uje), including a full memory dump.

Initial Triage

"Heavy CPU usage but no process can be seen consuming the CPU" sounds a lot like a rootkit hiding processes to me, but let's just confirm the SOC finding first. UAC collects the output from `"top -b -n1"` (`live_response/process/top_-b_-n1.txt`), and here's the first part of that file:

```
top - 19:38:21 up 15 min,  1 user,  load average: 4.38, 3.44, 1.81
Tasks: 132 total,   1 running, 131 sleeping,   0 stopped,   0 zombie
%Cpu(s): 97.7 us,  2.3 sy,   0.0 ni,   0.0 id,   0.0 wa,   0.0 hi,   0.0 si,
0.0 st
MiB Mem :   7947.4 total,   5206.4 free,   2780.6 used,    194.6
buff/cache
MiB Swap:   1101.0 total,   1097.7 free,     3.3 used.   5166.7 avail Mem
```

	PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
COMMAND											
	1	root	20	0	23812	14824	10756	S	0.0	0.2	0:00.70
systemd											
	2	root	20	0	0	0	0	S	0.0	0.0	0:00.00
kthreadd											
	3	root	20	0	0	0	0	S	0.0	0.0	0:00.00
pool_wor+											
[...]											

Looking at line #3, the CPU is indeed maxed out. The process listing below the header information should be sorted by CPU utilization, but we see nothing that remotely accounts for the amount of CPU being consumed. Seems like the SOC made a good read here.

UAC has some modules that look for common rootkit indicators, so we look in the “chkrootkit” directory in the UAC output and hit pay dirt (for those of you playing along at home, the [bodystat.sh](https://github.com/halpomeranz/dfis/blob/master/bodystat.sh) (<https://github.com/halpomeranz/dfis/blob/master/bodystat.sh>) script is available from my GitHub):

```
$ ls chkrootkit/
etc_ld_so_preload.txt  stat_etc_ld_so_preload.txt
$ cat chkrootkit/etc_ld_so_preload.txt
/lib/x86_64-linux-gnu/libymv.so.3
$ cat chkrootkit/stat_etc_ld_so_preload.txt
0|/etc/ld.so.preload|837578|-rw-r--r--
|0|0|34|1774394719|1774394709|1774394709|1774394709
$ cat chkrootkit/stat_etc_ld_so_preload.txt | bodystat.sh
File: /etc/ld.so.preload
Size: 34      UID: 0  GID: 0  Inode: 837578
Access: 2026-03-24 23:25:19
Modify: 2026-03-24 23:25:09
Change: 2026-03-24 23:25:09
Birth: 2026-03-24 23:25:09
```

```
$ grep -F lib/x86_64-linux-gnu/libymv.so.3 bodyfile/bodyfile.txt |
bodystat.sh
File: /usr/lib/x86_64-linux-gnu/libymv.so.3
Size: 24024  UID: 0  GID: 0  Inode: 715378
Access: 2026-03-24 23:25:19
Modify: 2026-03-24 23:24:51
Change: 2026-03-24 23:24:51
Birth: 2026-03-24 23:24:51
```

We have a suspicious library in `/etc/ld.so.preload` containing the library path `/usr/lib/x86_64-linux-gnu/libymv.so.3`. This `libymv.so.3` file was created at 2026-03-24 23:24:51 UTC and `/etc/ld.so.preload` at 23:25:09.

There are many directions our investigation could go from this point, but I got curious to see if we could tie those file creation times to a particular login session on the machine. A quick look at `live_response/system/last_i.txt` in the UAC data shows the following:

```
worker pts/0 192.168.4.35 Tue Mar 24 19:34 - still logged in
worker pts/0 192.168.4.35 Tue Mar 24 19:22 - 19:27 (00:04)
reboot system boot 6.12.74+deb13+1- Tue Mar 24 19:21 - still running
[...]
```

There's a time discrepancy here. If we assume that the session starting at 19:34 is where the UAC collection was made, this predates the arrival of the suspicious library by four hours. I suspect a local time zone issue.

```
$ ls -l \[root\]/etc/localtime
lrwxrwxrwx 1 hal hal 36 Mar 24 15:48 '[root]/etc/localtime' ->
/usr/share/zoneinfo/America/New_York
```

The “last” output will be in the default time zone for the machine, which is apparently US/Eastern time. That's four hours earlier than UTC at this time of year.

With that in mind, the creation times on our suspicious files line up nicely with the four minute session by user “worker” from 23:22 – 23:27 UTC (represented in the output above as 19:22 – 19:27). Unfortunately, UAC did not capture a `.bash_history` file for either the “worker” user or the “root” account. Anti-forensics is a possibility worth noting, but for now we don't have any useful command history.

Looking at the security logs should provide more details about that login session. The only logs available are the Systemd journal, so it's time to work some magic with `journalctl` (<https://righteousit.com/2024/08/06/systemd-journal-and-journalctl/>).

```
$ journalctl -D \[root\]/var/log/journal/ -q -o short-iso --
facility=auth,authpriv | grep -vE '(pam_unix|systemd-logind)'
[...]
```

2026-03-24T23:22:19+0000 vbox sshd-session[834]: Accepted password for worker from 192.168.4.35 port 48364 ssh2

2026-03-24T23:23:34+0000 vbox sudo[910]: worker : TTY=pts/0 ; PWD=/home/worker ; USER=root ; COMMAND=/bin/bash

2026-03-24T23:23:48+0000 vbox sshd-session[914]: Accepted password for worker from 192.168.4.35 port 55504 ssh2

2026-03-24T23:23:48+0000 vbox sshd-session[923]: Received disconnect from 192.168.4.35 port 55504:11: disconnected by user

2026-03-24T23:23:48+0000 vbox sshd-session[923]: Disconnected from user worker 192.168.4.35 port 55504

2026-03-24T23:25:19+0000 vbox sshd[801]: Received signal 15; terminating.

2026-03-24T23:25:19+0000 vbox sshd[937]: Server listening on 0.0.0.0 port 22.

2026-03-24T23:25:19+0000 vbox sshd[937]: Server listening on :: port 22.

2026-03-24T23:27:16+0000 vbox sshd-session[834]: syslogin_perform_logout: logout() returned an error

2026-03-24T23:27:16+0000 vbox sshd-session[873]: Received disconnect from 192.168.4.35 port 48364:11: disconnected by user

2026-03-24T23:27:16+0000 vbox sshd-session[873]: Disconnected from user worker 192.168.4.35 port 48364

[...]

The “worker” user logs in (with a password) at 23:22:19, and uses `sudo` to get a root shell at 23:23:34.

But at 23:23:48, we see a second SSH session for “worker” that did not appear in the “last” output. Typically this means that the session did not spawn an interactive shell and was not allocated a PTY. That would indicate that the session only ran a single command specified by the remote user on their command line, or was possibly an `scp`. This idea is supported by the fact that the session connected and disconnected in the same second.

At 23:25:19 we see the SSH server restarted. Curiouser and curiouser.

What We Know So Far

At this point we’re starting to build a timeline of the incident:

```
23:22:19    User "worker" logs in with password from jump host
(192.168.4.35) port 48364
23:23:34    User "worker" uses sudo to execute /bin/bash (root shell)
23:23:48    Command-only/scp as user "worker" from jump host (port 55504)
23:24:51    /usr/lib/x86_64-linux-gnu/libymv.so.3 created
23:25:09    /etc/ld.so.preload created (points to .../libymv.so.3)
23:25:19    SSH daemon restarted
23:27:16    User "worker" SSH session from jump host ends (source port
48364)
```

With everything laid out like this, it becomes obvious that the `/etc/ld.so.preload` file was created immediately before the SSH daemon was restarted. Since the only person operating on the system at the time was our suspected attacker, it is reasonable to assume that the SSH daemon was restarted so that it would pick up the suspicious `libymv.so.3` library.

`libymv.so.3` certainly has all the indications of being an LD_PRELOAD (https://righteousit.com/wp-content/uploads/2024/04/ld_preload-rootkits.pdf) type rootkit. Memory analysis is a good path for further investigation, so we will pick up there in our next installment.

Posted in [DFIR](#), [Forensics](#), [Linux](#) Tagged [Contest](#), [DFIR](#), [Forensics](#), [Linux](#), [Scenario](#)



Published by Hal Pomeranz

Independent Computer Forensics and Information Security consultant. Expert Witness. Trainer. [View all posts by Hal Pomeranz](#)

[Blog at WordPress.com.](#)